Modelling variability: feature models and UVL

REP · Reproducibility of computational experiments · Lab #3

INSA Rennes · 2026-2027

Mathieu Acher · mathieu.acher@irisa.fr · blog.mathieuacher.com

Notation adapted from the Software Product Lines course of TU Braunschweig (T. Thüm, T. Kehrer, E. Kuiter et al., CC-BY-SA-4.0), via the CPOO 2026-2027 material; these slides are released under the same license.

The lab #3 pipeline

```
fm/experiment.uvl        UVL Studio: write, count, configure
  | sample.py             script (flamapy): produces data
  ▼
configs/sample.csv       one row = one configuration
  | your script           template + compilation + CLI
  | CI matrix             version, architecture
  ▼
results/results.csv      one row = one run
  | analysis.ipynb        reads, produces nothing
  ▼
the factors that matter, and why
```

These slides: the first step, the `.uvl` file, and how to draw a sample from it.

A feature model describes a space of configurations

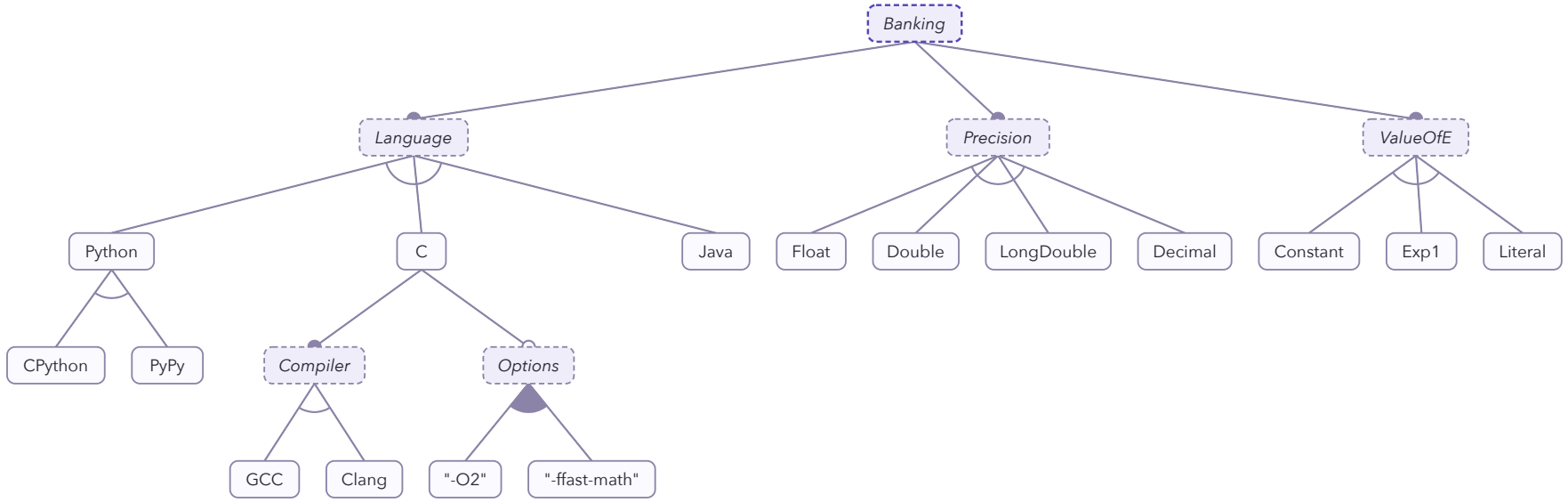
- a **feature**: a characteristic that a variant has, or does not have
- a **configuration**: the set of selected features, i.e. **one** variant
- a **feature model**: a **tree** of features + **constraints**, describing all the **valid** configurations, and only them

Running example, the banking problem (lab #1):

> You deposit e ≈ 2.71828… €. Each year n, the banker takes a 1 € fee, then multiplies by n. Withdrawing costs 1 €. How much do you get after 50 years?

A program that answers is **one** configuration: a language, an interpreter or a compiler, options, a precision, a way to get e…

The diagram



constraints

```
LongDouble => C
Decimal => Python | Java
Java & Decimal => !Exp1
```

● mandatory · ○ optional · empty arc: **alternative** (exactly one) · filled arc: **or** (at least one) · dashed box: **abstract**

Each construct has a meaning

Construct	UVL	If the parent is selected…	Banking
Mandatory ●	`mandatory`	the child is too	`Precision`; `Compiler` under `C`
Optional ○	`optional`	the child may be	`Options`
Alternative	`alternative`	**exactly one** child	`Language`: one language
Or	`or`	**at least one** child	`Options`: one option, or both
Cardinality	`[n..m]`	between n and m children	`[1..1]` = alternative, `[1..*]` = or

And always:

- a selected child ⇒ its parent is selected; the root is always selected
- `{abstract}`: a feature that only **structures** the tree (`Language`, `Compiler`…), drawn dashed

UVL: the text is the diagram

```
features
    Banking {abstract}
        mandatory
            Language {abstract}
                alternative
                    Python
                        alternative
                            CPython
                            PyPy
                    C
                        mandatory
                            Compiler {abstract}
                                alternative
                                    GCC
                                    Clang
                        optional
                            Options {abstract}
                                or
                                    "-O2"
                                    "-ffast-math"
                    Java
            Precision {abstract}
                ...
```

- two sections: `features`, then `constraints`
- **indentation** makes the tree (tabs or spaces: always the same)
- a **group** keyword between a parent and its children, never a child directly under a feature
- names with spaces or special characters: in **double quotes**, `"-O2"`
- `Language {abstract}`
- comments: `//` and `/* … */`

Cross-tree constraints

They relate features from **different branches** of the tree.

UVL	Meaning	Example
`A => B`	A **requires** B	`LongDouble => C`
`A => !B`	A **excludes** B	`Decimal => !C`
`A | B`	or	`Decimal => Python | Java`
`A & B`	and	`Java & Decimal => !Exp1`
`!A`	not	`!(Java & Decimal & Exp1)`
`A <=> B`	equivalence	`(A => B) & (B => A)`

Precedence, loosest to tightest: `<=>`, `=>`, `|`, `&`, `!`. `Java & Decimal => !Exp1` reads `(Java & Decimal) => (!Exp1)`.

> A constraint always has a reason: write it as a comment, `Java & Decimal => !Exp1 // BigDecimal has no exp()`.

Semantics: a Boolean formula

One variable per feature; the model is the **conjunction** of:

Construct	Formula
child `B` of `A` (always)	$B \Rightarrow A$
`B` **mandatory** under `A`	$A \Rightarrow B$
or group $B_1 \ldots B_n$ under `A`	$A \Rightarrow (B_1 \vee \ldots \vee B_n)$
alternative group	the same, + $\neg(B_i \wedge B_j)$ for all $i < j$
constraint	as is

Valid configuration = an assignment that makes the formula true:

Selection (+ the 4 abstract features at the top)	
`C`, `GCC`, `LongDouble`, `Exp1`	✅
`Python`, `CPython`, `PyPy`, `Double`, `Constant`	❌ one interpreter only
`Python`, `CPython`, `LongDouble`, `Constant`	❌ `LongDouble => C`
`C`, `GCC`, `Options`, `Float`, `Exp1`	❌ or: at least one option

Counting and analysing

An alternative **adds up** the configurations of its children; independent branches **multiply**:

Feature	Computation	Configurations
`C`	`Compiler` (2) × `Options` (1 + (2^2 − 1) = 4)	8
`Language`	`Python` + `C` + `Java` = 2 + 8 + 1	11
`Banking`	`Language` × `Precision` × `ValueOfE` = 11 × 4 × 3	**132**

The three constraints remove some of them: **98** valid configurations.

- **dead** feature: in no configuration. `"-ffast-math" => Java` (a bad copy-paste) kills it: 98 → 62
- **core** feature: in every configuration (`Banking`, `Language`, `Precision`, `ValueOfE`)
- **false-optional** feature: optional, yet present whenever its parent is
- **void** model: no valid configuration at all

The tools: UVL Studio, then `sample.py`

https://blog.mathieuacher.com/uvl-studio/: in the browser, nothing to install.

- editor with live diagnostics; **UVL cheat sheet** tab
- **configurator**: tick a feature, **propagation** does the rest
- **Analysis**: number of configurations, dead, core and false-optional features
- examples **Sandwich**, **Server**, **Pizza**; **Save .uvl**: the committed file is the reference

Then `sample.py` (flamapy, Python ≥ 3.11), on the same `.uvl` file:

```
python sample.py fm/experiment.uvl --size 30 --seed 1 -o configs/sample.csv
```

`--include mine.csvconf`: a configuration of your own, one line `Feature,True` per feature.

UVL: universal-variability-language.github.io · flamapy: flamapy.github.io